



Obrigado por se candidatar à WS Work, é uma honra ter você aqui!

Envie o código-fonte em formato .zip ou, preferencialmente, disponibilize-o em um repositório público no GitHub ou Bitbucket.

A aplicação deve utilizar:

- Python
- Framework web de sua preferência (FastAPI, Flask, Django, etc.)
- ORM para persistência de dados
- Banco de Dados relacional
- Endpoints REST
- JSON como formato de comunicação

Você poderá utilizar bibliotecas auxiliares que considerar apropriadas para melhorar a arquitetura e a experiência da aplicação.

1) Estrutura da Aplicação

Implemente a seguinte estrutura de entidades:

- **Marca**
 - id
 - nome_marca
- **Modelo**
 - id
 - marca_id
 - nome
 - valor_fipe
- **Carro**
 - id
 - modelo_id
 - timestamp_cadastro
 - ano
 - combustivel
 - num_portas
 - cor
 - quilometragem
 - valor_anuncio
 - descricao

2) Funcionalidade Inteligente (IA)

Além do CRUD tradicional, implemente uma funcionalidade que utilize IA ou processamento inteligente para agregar valor ao sistema.

Exemplos de funcionalidades:

- geração automática de descrição para venda;
- sugestão de preço baseado na Tabela FIPE;
- análise de valorização/depreciação;
- recomendação de veículos similares;
- classificação automática do veículo.

A solução pode utilizar integração com APIs externas ou lógica própria.

3) Endpoint para Consumo do Front-end

Crie um endpoint otimizado para consumo do front-end contendo a listagem de modelos em formato JSON.

A estrutura da resposta fica livre para proposta do candidato.

Implemente corretamente a estratégia de CORS considerando que o front-end poderá estar hospedado em outro domínio.

4) CRUD e Persistência

Implemente endpoints completos para:

- Carros
- Marcas
- Modelos

A persistência deve utilizar ORM e banco de dados relacional.

5) Documentação e Organização

Documente a aplicação utilizando Swagger/OpenAPI ou ferramenta equivalente.

Explique brevemente:

- estrutura do projeto;
- organização das camadas;
- decisões técnicas;
- estratégia utilizada para IA;
- como executar a aplicação;
- como consumir os endpoints.

6) Diferenciais (Opcional)

Será considerado diferencial:

- Docker/Docker Compose;
- testes automatizados;
- boas práticas de arquitetura;
- autenticação/autorização;
- deploy da aplicação;
- utilização de tipagem e boas práticas Python.

Requisitos Gerais

- O projeto deve compilar e executar corretamente.
- O código deve estar organizado e legível.
- Utilize boas práticas de desenvolvimento.
- Fique à vontade para propor melhorias na estrutura e arquitetura da solução.

Boa Sorte!